

任务号	4
姓名	杨力泉 元杰
学号	231220069 231220082
日期	2026 年 6 月 2 日
附加任务	无（仅完成必做部分）

实验报告 — 目标代码生成器实现与调试

一、实验目标与概述

在前三个实验基础上，实现 C- 中间代码到 MIPS32 汇编的翻译器——编译器后端的代码生成器。程序读取符合实验三规范的三地址码 IR，输出可在 SPIM Simulator 上运行的 MIPS32 汇编代码。最后阶段结合中间代码翻译实现完整的两阶段流水线：C- 源码 → IR 临时文件 → .s 汇编文件。

二、总体设计与数据结构

- IR 解析**：逐行 `fgets` + 空白分隔 `tokenize`，识别 `FUNCTION`、`PARAM`、`x := ...`、`IF ... GOTO ...` 等 16 种 IR 指令，存入 IR 结构体（`IROp` 枚举 + 最多 4 个操作数字段 `a[4][64]`）。
- 栈帧管理**：每个函数维护 `Frame` 结构，记录形参列表、局部变量列表、各变量相对 `$sp` 的偏移量、总帧大小 `frame_size` 及 `has_calls` 标志（控制 `$ra` 保存）。
- 寄存器分配**：朴素方案——全部变量在栈上分配 4 字节空间，运算时 `lw $t0/$t1` 加载，算术运算后 `sw $t0` 写回。调用参数预加载到 `$t4-$t7`，调整 `$sp` 后再 `move $aN, $tN`，避免 `$sp` 偏移失效。

三、核心实现

3.1 指令选择（IR → MIPS32）

IR 指令	MIPS32 翻译
<code>x := #k</code>	<code>li \$t0, k → sw \$t0, off_x(\$sp)</code>
<code>x := y</code>	<code>lw \$t0, off_y(\$sp) → sw \$t0, off_x(\$sp)</code>
<code>x := y + z</code>	<code>lw \$t0, y; lw \$t1, z; addu \$t0,\$t0,\$t1; sw \$t0, x</code>
<code>x := y + #k</code>	<code>lw \$t0, y; addiu \$t0,\$t0,k; sw \$t0, x</code> （常量折叠）
<code>x := y - z / * //</code>	<code>lw \$t0, y; lw \$t1, z; subu/mul/div \$t0,\$t0,\$t1; sw \$t0, x</code>
<code>x := *y</code>	<code>lw \$t0, y; lw \$t0, 0(\$t0); sw \$t0, x</code>
<code>*x := y</code>	<code>lw \$t0, x; lw \$t1, y; sw \$t1, 0(\$t0)</code>
<code>LABEL L: / GOTO L</code>	<code>L: / j L</code>
<code>IF x relop y GOTO L</code>	<code>lw \$t0, x; lw \$t1, y; b<relop> \$t0,\$t1,L</code> （6 种 <code>relop</code> 查表映射）
<code>RETURN x</code>	<code>lw \$t0, x; move \$v0,\$t0 → Epilogue → jr \$ra</code>

IR 指令	MIPS32 翻译
READ x	jal read → sw \$v0, off_x(\$sp)
WRITE x	lw \$t0, x; move \$a0,\$t0; jal write
x := CALL f	收集 ARG → 调用序列 → jal f → sw \$v0, off_x(\$sp)

3.2 函数调用序列与栈管理

调用者 (do_call)：预加载实参到 \$t4-\$t7 → addiu \$sp, -N; sw \$ra, N-4(\$sp) → move \$a0-\$a3, \$t4-\$t7 → jal f → lw \$ra; addiu \$sp, N → sw \$v0 到目标变量。

被调用者 Prologue：addiu \$sp, -framesize; sw \$ra → 复制 \$a0-\$a3 到形参栈槽。Epilogue：move \$v0 → lw \$ra; addiu \$sp, framesize; jr \$ra。

3.3 预定义函数

read/write 以 MIPS 汇编硬编码在 .text 段：read 通过 syscall 4 (print_string 输出提示) → syscall 5 (read_int 读整数)；write 通过 syscall 1 (print_int) → syscall 4 (print_string 输出换行)，返回值固定为 0。

3.4 个人实现

1. **常量折叠**：x := y + #k 和 x := y - #k 直接生成 addiu 指令，省去一次 lw 和一次寄存器分配。
2. **模块化 IR 解析**：独立 parse_ir() 函数将文本 IR 转为结构化 IR[] 数组，generate_all() 按函数边界遍历生成代码，结构清晰。

四、遇到的主要困难与解决方案

1. **调用中 \$sp 偏移错误**：先 addiu \$sp 再 lw 加载参数时，偏移量是相对于旧 \$sp 的 → 改为预加载到 \$t4-\$t7，调整 \$sp 后 move \$aN, \$tN。
2. **变量重复分配栈空间**：add_var 对同一变量多次调用 → 添加 find_var 去重检查。
3. **栈帧大小对齐**：帧大小需 8 字节对齐且 has_calls 时额外预留 \$ra 槽位 → 统一在 Prologue/Epilogue 中计算 fs = frame_bytes + (has_calls ? 4 : 0)。

五、构建与运行

```
cd Lab4.0/Code
make                                # 编译生成 parser
./parser input.cmm output.s         # 翻译 C-- → MIPS32
spim -file output.s                 # 在 SPIM 中运行
```

六、总结与后续工作

本次实验完成了 C-- 编译器后端的完整实现——从三地址码到 MIPS32 汇编的翻译器，涵盖必做全部功能：16 种 IR 指令选择、朴素寄存器分配、完整栈帧管理和函数调用序列。核心工作是设计正确的函数调用时序（预加载 → 调栈 → 传参 → jal → 恢复）和 IR 解析器。